Homework Reflections 5–8

Student: Gaurav Kumar **Course:** DX702O1

(Week 5)

Question 1

> *Draw a diagram for the following negative feedback loop: Sweating causes body temperature to decrease. High body temperature causes sweating. A negative feedback loop means that one thing increases another while the second thing decreases the first. Remember that we are using directed acyclic graphs where two things cannot directly cause each other.*

A direct drawing of this loop would be Temperature → Sweating → Temperature, which is a cycle, and a DAG cannot have two variables cause each other. I removed the cycle by indexing each variable by time. High body temperature at time t increases sweating at time t (a positive edge). Sweating at time t then decreases body temperature at the next time step, t+1 (a negative edge). I also added an edge Temp(t) → Temp(t+1), because body temperature carries over from one moment to the next.

The feedback is negative because the two edges in each round have opposite signs: (+) × (−) = (−). If temperature rises, sweating rises, which pushes the next temperature back down. This stabilizes the system around a set point, which is how the body maintains homeostasis. Every arrow points forward in time, so the graph is acyclic; I confirmed this with `networkx.is_directed_acyclic_graph`.

```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()
edges = [
    ('Temp(t0)',  'Sweat(t0)', 'increases (+)'),   # high temperature causes sweating
    ('Sweat(t0)', 'Temp(t1)',  'decreases (-)'),   # sweating lowers the NEXT temperature
    ('Temp(t0)',  'Temp(t1)',  'persists (+)'),    # temperature carries over in time
    ('Temp(t1)',  'Sweat(t1)', 'increases (+)'),
    ('Sweat(t1)', 'Temp(t2)',  'decreases (-)'),
    ('Temp(t1)',  'Temp(t2)',  'persists (+)'),
]
for u, v, label in edges:
    G.add_edge(u, v, label=label)

print("Is DAG:", nx.is_directed_acyclic_graph(G))   # True -> no cycles

pos = {'Temp(t0)': (0, 1), 'Sweat(t0)': (1, 0), 'Temp(t1)': (2, 1),
       'Sweat(t1)': (3, 0), 'Temp(t2)': (4, 1)}
colors = ['tomato' if n.startswith('Temp') else 'skyblue' for n in G]

plt.figure(figsize=(10, 4))
nx.draw(G, pos, with_labels=True, node_color=colors, node_size=2800,
        font_size=9, arrowsize=20)
nx.draw_networkx_edge_labels(G, pos, edge_labels=nx.get_edge_attributes(G, 'label'),
                             font_size=9)
plt.title('Negative feedback loop unrolled over time (acyclic)')
plt.show()
```

Code — Week 5 Question 1

Question 2

> *Describe an example of a positive feedback loop. This means that one thing increases another while the second thing also increases the first.*

A classic positive feedback loop is the ice–albedo effect in climate. Ice and snow are bright and reflect most incoming sunlight back to space. When global temperature rises, some ice melts and exposes darker ocean or land underneath. Darker surfaces absorb more sunlight, which raises temperature further, which melts more ice. Each variable increases the other: Temperature → (+) Ice melt, and Ice melt → (+) Temperature.

As with the sweating example, this can't be drawn as Temperature ⇄ Ice melt in a DAG, because two variables cannot directly cause each other. So I unrolled it over time: Temp(t) → IceMelt(t) → Temp(t+1). Both edges are positive, so the product of signs is (+) × (+) = (+),

and a small push grows instead of fading. That is the key contrast with Question 1: negative feedback (sweating) pulls the system back to a set point, while positive feedback amplifies a change until something outside the loop stops it. Here, that limit is running out of ice.

```
G2 = nx.DiGraph()
edges2 = [
    ('Temp(t0)',    'IceMelt(t0)', 'increases (+)'),   # warmer -> more ice melts
    ('IceMelt(t0)', 'Temp(t1)',    'increases (+)'),   # less ice -> less reflection -> warmer
    ('Temp(t0)',    'Temp(t1)',    'persists (+)'),
    ('Temp(t1)',    'IceMelt(t1)', 'increases (+)'),
    ('IceMelt(t1)', 'Temp(t2)',    'increases (+)'),
    ('Temp(t1)',    'Temp(t2)',    'persists (+)'),
]
for u, v, label in edges2:
    G2.add_edge(u, v, label=label)

print("Is DAG:", nx.is_directed_acyclic_graph(G2))

pos2 = {'Temp(t0)': (0, 1), 'IceMelt(t0)': (1, 0), 'Temp(t1)': (2, 1),
        'IceMelt(t1)': (3, 0), 'Temp(t2)': (4, 1)}
colors2 = ['tomato' if n.startswith('Temp') else 'lightblue' for n in G2]

plt.figure(figsize=(10, 4))
nx.draw(G2, pos2, with_labels=True, node_color=colors2, node_size=2800,
        font_size=9, arrowsize=20)
nx.draw_networkx_edge_labels(G2, pos2, edge_labels=nx.get_edge_attributes(G2, 'label'),
                             font_size=9)
plt.title('Positive feedback loop (ice-albedo) unrolled over time (acyclic)')
plt.show()
```

Code — Week 5 Question 2 (DAG)

```
import numpy as np
np.random.seed(0)
steps = 30

def simulate(sign):
    temp = np.zeros(steps)
    temp[0] = 0.1                                          # small initial warming
    for t in range(1, steps):
        melt = 0.5 * temp[t-1] + np.random.normal(0, 0.02)  # warmer -> more melt
        temp[t] = temp[t-1] + sign * 0.3 * melt + np.random.normal(0, 0.02)
    return temp

plt.figure(figsize=(8, 3.5))
plt.plot(simulate(+1), label='Positive loop: melt raises temperature')
plt.plot(simulate(-1), label='Negative loop (like sweating)')
plt.xlabel('time step'); plt.ylabel('temperature anomaly'); plt.legend()
plt.title('Positive feedback amplifies, negative feedback stabilizes')
plt.show()
```

Code — Week 5 Question 2 (simulation of amplification vs. stabilization)

Question 3

> *Draw a diagram for the following situation: Lightning storms frighten away deer and bears, decreasing their population, and cause flowers to grow, increasing their population. Bears eat deer, decreasing their population. Deer eat flowers, decreasing their population. Write a dataset that simulates this situation. (Show the code.) Include noise / randomness in all cases. Identify a backdoor path with one or more confounders for the relationship between deer and flowers.*

The diagram has three paths between Deer and Flowers. The direct causal path is Deer → Flowers (deer eat flowers). The main backdoor path is Deer ← Lightning → Flowers, where Lightning is a confounder: storms lower the deer population and also raise the flower population. A second backdoor path is Deer ← Bears ← Lightning → Flowers. Both backdoor paths run through Lightning.

In the simulation, the true effect of one more deer is −0.8 flowers. A naive regression of flowers on deer gives about −1.72, more than double the true effect. That's because in seasons with few deer there were usually many storms, and storms also raise flower counts, so deer look more harmful than they are. Adding lightning to the regression closes both backdoor paths and gives about −0.73, close to the true −0.8. Controlling for bears alone does not help (−1.80), because the path Deer ← Lightning → Flowers is still open.

```
G3 = nx.DiGraph()
edges3 = [
    ('Lightning', 'Deer',    'decreases (-)'),   # storms frighten deer away
    ('Lightning', 'Bears',   'decreases (-)'),   # storms frighten bears away
    ('Lightning', 'Flowers', 'increases (+)'),   # storms help flowers grow
    ('Bears',     'Deer',    'decreases (-)'),   # bears eat deer
    ('Deer',      'Flowers', 'decreases (-)'),   # deer eat flowers
]
for u, v, label in edges3:
    G3.add_edge(u, v, label=label)

print("Is DAG:", nx.is_directed_acyclic_graph(G3))

pos3 = {'Lightning': (1, 2), 'Bears': (0, 1), 'Deer': (1, 0), 'Flowers': (2, 1)}
colors3 = {'Lightning': 'gold', 'Bears': 'saddlebrown', 'Deer': 'peru', 'Flowers': 'violet'}

plt.figure(figsize=(7, 5))
nx.draw(G3, pos3, with_labels=True, node_color=[colors3[n] for n in G3],
        node_size=3000, font_size=10, arrowsize=20)
nx.draw_networkx_edge_labels(G3, pos3, edge_labels=nx.get_edge_attributes(G3, 'label'),
                             font_size=9)
plt.title('Lightning, bears, deer and flowers (DAG)')
plt.show()
```

Code — Week 5 Question 3 (DAG)

```
# Step 2 : Simulated dataset
import pandas as pd

np.random.seed(42)
n = 1000

lightning = np.random.poisson(3, n)                                    # storms per season (random)
bears     = 50  - 2 * lightning               + np.random.normal(0, 3, n)
deer      = 200 - 5 * lightning - 1.5 * bears  + np.random.normal(0, 10, n)
flowers   = 500 + 20 * lightning - 0.8 * deer  + np.random.normal(0, 25, n)

eco = pd.DataFrame({'lightning': lightning, 'bears': bears,
                    'deer': deer, 'flowers': flowers})
eco.head()
```

Code — Week 5 Question 3 (simulated dataset with noise)

```
import statsmodels.formula.api as smf

# All paths between Deer and Flowers (ignoring arrow direction)
for path in nx.all_simple_paths(G3.to_undirected(), 'Deer', 'Flowers'):
    print(' - '.join(path))

naive    = smf.ols('flowers ~ deer', eco).fit().params['deer']
adjusted = smf.ols('flowers ~ deer + lightning', eco).fit().params['deer']
print(f"True effect of deer on flowers: -0.800")
print(f"Naive (no adjustment):          {naive:.3f}")
print(f"Adjusting for lightning:        {adjusted:.3f}")
```

Code — Week 5 Question 3 (regressions showing the backdoor path)

Question 4

> *Draw a diagram for a situation of your own invention. The diagram should include at least four nodes, one confounder, and one collider. Be sure that it is acyclic (no loops). Which node would you say is most like a treatment (X)? Which is most like an outcome (Y)?*

My example asks whether attending **tutoring** raises a student's **exam score**. The graph has five nodes. **Motivation** is the confounder: motivated students are more likely to sign up for tutoring (Motivation → Tutoring), and they also score higher on their own (Motivation → ExamScore). That makes a backdoor path, Tutoring ← Motivation → ExamScore. **StudyHours** is a mediator on the real causal path: Tutoring → StudyHours → ExamScore. **Honors** (admission to an honors program) is the collider: a student can get in through a tutor's recommendation or through a high exam score, so two arrows point into it (Tutoring → Honors ← ExamScore). Every arrow goes forward from cause to effect and none returns to an earlier node, so the graph is acyclic. `nx.is_directed_acyclic_graph` confirms this.

Tutoring is the treatment (X), because it is the choice or intervention we would like to evaluate. **ExamScore is the outcome (Y)**, because it is the downstream result we care about. Motivation and Honors are neither. Motivation is a common *cause* of X and Y, and Honors is a common *effect* of X and Y.

The two special nodes need opposite handling. To remove confounding bias, we **should control for the confounder** (Motivation), which blocks the backdoor path. We **should not control for the collider** (Honors). A collider blocks its path on its own, but conditioning on it opens that path. For example, among honors students only, someone who got in without a high score most likely got in through tutoring. This creates a false negative link between tutoring and score. The simulation below shows both effects. The true effect is 6 points. The naive estimate is about 11 because motivation inflates it. Controlling for Motivation gives about 6.

Adding Honors as a control, or keeping only honors students, pulls the estimate down to about 3.

```
G4 = nx.DiGraph()
edges4 = [
    ('Motivation', 'Tutoring',   'confounder path'),  # motivated students sign up for tutoring
    ('Motivation', 'ExamScore',  'confounder path'),  # ...and also score higher on their own
    ('Tutoring',   'StudyHours', 'causal'),           # tutoring builds a study routine
    ('StudyHours', 'ExamScore',  'causal'),           # more study -> higher score
    ('Tutoring',   'Honors',     'collider'),         # tutor writes the honors recommendation
    ('ExamScore',  'Honors',     'collider'),         # high score also earns honors admission
]
for u, v, label in edges4:
    G4.add_edge(u, v, label=label)

print("Is DAG:", nx.is_directed_acyclic_graph(G4))   # True -> no loops

# A confounder has arrows INTO both X and Y; a collider has arrows coming in from both X and Y
X, Y = 'Tutoring', 'ExamScore'
confounders = [n for n in G4 if G4.has_edge(n, X) and G4.has_edge(n, Y)]
colliders   = [n for n in G4 if G4.has_edge(X, n) and G4.has_edge(Y, n)]
print("Confounder(s):", confounders)
print("Collider(s):  ", colliders)

pos4 = {'Motivation': (2, 2), 'Tutoring': (0, 1), 'StudyHours': (2, 1),
        'ExamScore': (4, 1), 'Honors': (2, 0)}
role = {'Motivation': 'gold', 'Tutoring': 'lightgreen', 'StudyHours': 'lightgray',
        'ExamScore': 'tomato', 'Honors': 'plum'}
colors4 = [role[n] for n in G4]

plt.figure(figsize=(10, 5))
nx.draw(G4, pos4, with_labels=True, node_color=colors4, node_size=3200,
        font_size=9, arrowsize=20)
nx.draw_networkx_edge_labels(G4, pos4, edge_labels=nx.get_edge_attributes(G4, 'label'),
                             font_size=8)
plt.title('Tutoring (X) -> Exam score (Y): Motivation = confounder, Honors = collider')
plt.show()
```

Code — Week 5 Question 4 (DAG construction and role detection)

```
rng = np.random.default_rng(4)
n = 10_000

motivation  = rng.normal(0, 1, n)
tutoring    = (motivation + rng.normal(0, 1, n) > 0).astype(float)    # 1 = attended tutoring
study_hours = 5 + 3 * tutoring + rng.normal(0, 1, n)                  # tutoring adds ~3 hrs/week
exam_score  = 60 + 2 * study_hours + 5 * motivation + rng.normal(0, 3, n)
honors      = (2 * tutoring + 0.2 * (exam_score - 70) + rng.normal(0, 1, n) > 2).astype(float)

def tutoring_coef(*controls, mask=None):
    """OLS coefficient on tutoring, optionally controlling for other variables / subsetting rows."""
    m = np.ones(n, bool) if mask is None else mask
    A = np.column_stack([np.ones(m.sum()), tutoring[m]] + [c[m] for c in controls])
    return np.linalg.lstsq(A, exam_score[m], rcond=None)[0][1]

print("True total effect of tutoring:                6.00  (3 hrs x 2 pts/hr)")
print(f"Naive (no controls):                          {tutoring_coef():.2f}  <- confounded")
print(f"Control for Motivation (blocks backdoor):     {tutoring_coef(motivation):.2f}  <- correct")
print(f"Control for Motivation + Honors (collider):   {tutoring_coef(motivation, honors):.2f}  <- collider bias")
print(f"Only honors students, control for Motivation: {tutoring_coef(motivation, mask=honors == 1):.2f}  <- collider bias")
```

Code — Week 5 Question 4 (confounder vs. collider simulation)

(Week 6)

Question 1

> *What is a potential problem with computing the Marginal Treatment Effect simply by comparing each untreated item to its counterfactual and taking the maximum difference? (Hint: think of statistics here. Consider that only the most extreme item ends up being used to estimate the MTE. That's not necessarily a bad thing; the MTE is supposed to come from the untreated item that will produce the maximum effect. But there is nevertheless a problem.)*

The problem is that the maximum of many noisy estimates is a biased and unstable statistic. Each student's estimated effect, $Y(1) - Y(0)$, equals their true effect plus random noise. When we take the maximum over thousands of students, we pick whoever had the most extreme noise draw, not necessarily the person with the largest true effect. This is the "winner's curse" (also called regression to the mean, or a multiple-comparisons problem): the more items we compare, the more likely one of them looks extreme purely by chance.

I demonstrated this with the tutoring model from Week 5, where tutoring raises every student's score by exactly 6 points, so the true marginal treatment effect is 6. Yet the largest estimated effect among 4,912 untreated students was 23.6, almost four times the truth. Across 200 reruns, the maximum ranged from 21.4 to 29.9, so the estimate also has high variance. The maximum also keeps growing with sample size (about 17.7 at n = 100 and 27.2 at n = 100,000) even though the true effect never changes. A useful estimator should converge to the true value as we collect more data; the maximum moves away from it.

So the naive MTE is driven by a single noisy observation. It is biased upward, sensitive to sample size, and not reproducible. If we targeted that "most responsive" student, their next outcome would most likely fall back toward 6. A more robust approach is to estimate a high

quantile of the effect distribution, or to average the top group of students, instead of relying on one extreme point.

```python
import numpy as np

def untreated_effects(seed, n=10_000):
    """Same tutoring world as Week 5, but generate BOTH potential outcomes for everyone."""
    rng = np.random.default_rng(seed)
    motivation = rng.normal(0, 1, n)
    tutoring   = (motivation + rng.normal(0, 1, n) > 0).astype(float)

    # Potential outcomes: same person, with and without tutoring (each with its own noise)
    study0 = 5     + rng.normal(0, 1, n)
    study1 = 5 + 3 + rng.normal(0, 1, n)
    y0 = 60 + 2 * study0 + 5 * motivation + rng.normal(0, 3, n)
    y1 = 60 + 2 * study1 + 5 * motivation + rng.normal(0, 3, n)

    return (y1 - y0)[tutoring == 0]           # individual effects for the untreated

d = untreated_effects(4)
print(f"Untreated students:          {d.size}")
print(f"True effect for EVERYONE:     6.00")
print(f"Mean estimated effect:        {d.mean():.2f}")
print(f"MAX estimated effect ('MTE'): {d.max():.2f}   <- the naive MTE")

# 1) The max is unstable: rerun the same world with different seeds
maxes = [untreated_effects(s).max() for s in range(200)]
print(f"\nMax across 200 reruns: mean {np.mean(maxes):.2f}, sd {np.std(maxes):.2f}, "
      f"range {min(maxes):.1f} - {max(maxes):.1f}")

# 2) The max grows with sample size even though the true effect never changes
for size in [100, 1_000, 10_000, 100_000]:
    print(f"n = {size:>7,}: average max = {np.mean([untreated_effects(s, size).max() for s in range(50)]):.2f}")
```

Code — Week 6 Question 1 (winner's curse demonstration)

Question 2

> *Propose a solution that remedies this problem and write some code that implements your solution. It's very important here that you clearly explain what your solution will do.*

My solution replaces the maximum estimated effect with the **90th percentile** of the estimated effects across all untreated items. The code works like this. It creates 200 untreated items, each with a fixed true treatment effect. In each of 1,000 repetitions, it estimates each item's counterfactual with noise (SD = 1), just as a real model would. Each repetition then records two summaries: the largest estimated effect (the original MTE method) and the 90th percentile of the estimated effects (my proposal). Because I know the true effects, I can measure how far off each method is on average (bias) and how much it jumps around between repetitions (SD).

The results show the problem from Question 1. The max of the estimates averages 5.96, while the true MTE is 5.05, so it overshoots by about 0.9. That's because the item that "wins" is usually one whose noise happened to be large and positive. The max is also unstable (SD = 0.56), because it depends on a single item. The 90th percentile is an order statistic in the middle of the upper tail, so it combines information from many items and no single lucky draw controls it. Its SD is 0.15, almost four times smaller.

The trade-off is that the 90th percentile answers a slightly different question: "what effect would a high-response item get?" rather than "what is the largest effect?" It is still a little inflated (+0.57), because noise spreads out the whole distribution of estimates. A further fix would be to first shrink each estimate toward the overall mean, based on how noisy it is, and then take the percentile. For a decision such as "which untreated items should we treat next?", a stable estimate for the top 10% is more useful than an unreliable estimate for the single best item.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(6)
n_items, n_reps = 200, 1000
noise_sd = 1.0                    # error in estimating each item's counterfactual

true_effect = rng.normal(2, 1, n_items)          # fixed true effect for each untreated item
true_mte = true_effect.max()                     # target: the biggest true effect
true_p90 = np.percentile(true_effect, 90)

max_est, p90_est = [], []
for _ in range(n_reps):
    # each repetition = a new noisy estimate of every item's counterfactual
    est_effect = true_effect + rng.normal(0, noise_sd, n_items)
    max_est.append(est_effect.max())
    p90_est.append(np.percentile(est_effect, 90))
max_est, p90_est = np.array(max_est), np.array(p90_est)

print(f"True MTE (max true effect):      {true_mte:.2f}")
print(f"True 90th percentile of effects: {true_p90:.2f}\n")
print(f"{'method':<22}{'mean':>7}{'target':>8}{'bias':>7}{'SD':>7}")
for name, est, target in [("max of estimates", max_est, true_mte),
                          ("90th pct of estimates", p90_est, true_p90)]:
    print(f"{name:<22}{est.mean():>7.2f}{target:>8.2f}{est.mean() - target:>+7.2f}{est.std():>7.2f}")

plt.figure(figsize=(8, 3.5))
plt.hist(max_est, bins=40, alpha=0.6, label='max of estimates')
plt.hist(p90_est, bins=40, alpha=0.6, label='90th percentile of estimates')
plt.axvline(true_mte, color='k', ls='--', label='true MTE')
plt.axvline(true_p90, color='gray', ls=':', label='true 90th percentile')
plt.xlabel('estimated effect'); plt.ylabel('count'); plt.legend(fontsize=8)
plt.title('Max is inflated and noisy; 90th percentile is far more stable')
plt.show()
```

Code — Week 6 Question 2 (90th-percentile vs. max comparison)

(Week 7)

Question 1

> *Create a linear regression model involving a confounder that is left out of the model. Show whether the true correlation between X and Y is overestimated, underestimated, or neither. Explain in words why this is the case for the given coefficients you have chosen.*

I simulated a confounder W that causes both the treatment and the outcome: $X = 0.8 \cdot W +$ noise and $Y = 2 \cdot X + 3 \cdot W +$ noise. The true effect of X on Y is 2. When W is included in the regression, the X coefficient is 1.98, which matches the truth. When W is left out, the X coefficient jumps to 3.46, so the effect is **overestimated** by about 73%.

The reason is that X and W move together (W raises X by 0.8 for each unit of W). Once W is removed from the model, the regression gives X credit for W's effect on Y, because high-X units also tend to have high W. The omitted-variable bias formula states this exactly: bias = (effect of W on Y) × Cov(X, W)/Var(X) = $3 \times 0.8/1.64 \approx 1.46$, and $2 + 1.46 = 3.46$, which matches the simulation.

The direction of the bias depends on the signs. Here both of W's effects are positive (W raises X and raises Y), so the bias is positive and X looks more effective than it is. If I change W's effect on Y to −3, the two signs disagree and the X coefficient drops to 0.54, an underestimate. Only when W is unrelated to X, or has no effect on Y, does leaving it out cause no bias.

```python
import numpy as np
import statsmodels.api as sm

rng = np.random.default_rng(7)
n = 10_000

# W is the confounder: it causes both X and Y
W = rng.normal(0, 1, n)
X = 0.8 * W + rng.normal(0, 1, n)           # W raises X
Y = 2 * X + 3 * W + rng.normal(0, 1, n)     # true effect of X on Y = 2; W also raises Y

full    = sm.OLS(Y, sm.add_constant(np.column_stack([X, W]))).fit()   # includes W
omitted = sm.OLS(Y, sm.add_constant(X)).fit()                         # W left out

# Omitted-variable bias formula: b_X = true_b_X + b_W * Cov(X, W) / Var(X)
predicted = 2 + 3 * 0.8 / (0.8**2 + 1)

print(f"True effect of X on Y:          2.000")
print(f"Model with W (correct):         {full.params[1]:.3f}")
print(f"Model without W (confounded):   {omitted.params[1]:.3f}  <- overestimated")
print(f"Predicted by the OVB formula:   {predicted:.3f}")

# Same setup, but W now LOWERS Y -> the bias reverses
Y_neg = 2 * X - 3 * W + rng.normal(0, 1, n)
print(f"\nIf W lowers Y instead (-3 * W): {sm.OLS(Y_neg, sm.add_constant(X)).fit().params[1]:.3f}  <- underestimated")
```

Code — Week 7 Question 1 (omitted-variable bias demonstration)

Question 2

> *Perform a linear regression analysis in which one of the coefficients is zero, e.g. W = [noise] X = [noise] Y = 2 * X + [noise] And compute the p-value of a coefficient — in this case, the coefficient of W. Run the analysis 1000 times and report the best (smallest) p-value. If the p-value is less than 0.05, does this mean the coefficient actually is nonzero? What is the problem with repeating the analysis?*

I simulated W and X as independent standard-normal noise, with Y = 2X + noise and n = 100. That means the true coefficient on W is exactly zero. I then fit the regression Y ~ W + X 1,000 times, each time on fresh data, and recorded the p-value for W's coefficient. The **smallest p-value was 0.0028**, with an estimated W coefficient of +0.31. Taken alone, that looks like strong evidence that W affects Y. In addition, 46 of the 1,000 runs (4.6%) had $p < 0.05$.

No, a p-value below 0.05 does not mean the coefficient is actually nonzero. We know the truth here: W has no effect on Y. A p-value is the probability of an estimate at least this extreme *assuming* the true coefficient is zero. So when the null hypothesis is true, p-values are uniformly distributed between 0 and 1, which is what the flat histogram shows. About 5% of

runs will fall below 0.05 purely by chance; these are false positives (Type I errors). The 0.05 threshold controls the error rate for **one** pre-planned test, not for the best of many tests.

The problem with repeating the analysis is **multiple testing**, also called p-hacking. Each run has a 5% chance of a false positive. The chance that *none* of the 1,000 runs falls below 0.05 is 0.95^1000 ≈ 5×10^{-23}. So reporting the smallest p-value almost guarantees a "significant" result, even when there is no effect at all. The same thing happens in practice when someone tries many specifications, subgroups, or datasets and reports only the one that worked. A fix is to pre-register a single test, or to correct for the number of tests. For example, with a Bonferroni correction (0.05 / 1000 = 0.00005), none of the 1,000 runs is significant, which is the correct conclusion.

```python
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

rng = np.random.default_rng(7)
n, n_runs = 100, 1000

p_values, w_coefs = [], []
for _ in range(n_runs):
    W = rng.normal(0, 1, n)                  # pure noise, no effect on Y
    X = rng.normal(0, 1, n)
    Y = 2 * X + rng.normal(0, 1, n)          # true coefficient of W is 0

    design = sm.add_constant(np.column_stack([W, X]))   # columns: const, W, X
    fit = sm.OLS(Y, design).fit()
    w_coefs.append(fit.params[1])
    p_values.append(fit.pvalues[1])
p_values, w_coefs = np.array(p_values), np.array(w_coefs)

best = p_values.argmin()
print(f"Smallest p-value for W:       {p_values[best]:.5f}  (W coef = {w_coefs[best]:+.3f})")
print(f"Runs with p < 0.05:           {(p_values < 0.05).sum()} / {n_runs}  "
      f"({(p_values < 0.05).mean():.1%}, expected ~5%)")
print(f"P(no p < 0.05 in {n_runs} runs) = 0.95^{n_runs} = {0.95**n_runs:.1e}  "
      f"-> a 'significant' result is virtually guaranteed")
print(f"Bonferroni threshold 0.05/{n_runs} = {0.05/n_runs:.5f} -> "
      f"runs still 'significant': {(p_values < 0.05/n_runs).sum()}")

plt.figure(figsize=(8, 3.5))
plt.hist(p_values, bins=20, range=(0, 1), edgecolor='white')
plt.axvline(0.05, color='red', ls='--', label='p = 0.05')
plt.xlabel('p-value of W'); plt.ylabel('count'); plt.legend()
plt.title('When the true coefficient is 0, p-values are uniform: ~5% fall below 0.05 by chance')
plt.show()
```

Code — Week 7 Question 2 (p-hacking with 1000 reruns)

(Week 8)

Question 1

> *Include the code you used to solve Week 8's coding quiz problems and write about the obstacles / challenges / insights you encountered while solving them.*

Approach and choices. For Questions 1–2 of the quiz, I estimated the ATE with inverse probability weighting. A logistic regression of X on Z gave each row's propensity score p = P(X=1 | Z), and each row was weighted by 1/p if treated or 1/(1−p) if untreated. I made three choices. First, I set `C=np.inf` to turn off scikit-learn's default L2 penalty, which shrinks coefficients and pulls the scores toward 0.5; we want the plain probability here. Second, I used normalized (Hájek) weighted means via `np.average` rather than the unnormalized Horvitz-Thompson formula, because normalized weights are more stable when a few weights are large. Third, I checked overlap before trusting the result. The IPW ATE was ≈ **2.27**, compared with a naive difference of 3.04. The first three propensity scores were **0.842, 0.585, 0.712**. For quiz Questions 3–4, I matched each treated row to its nearest untreated row by Mahalanobis distance, with replacement, which gave an **ATT ≈ 3.44** compared with a naive 4.05. The treated item with the least common support had (Z1, Z2) = (2.696, 0.538), and its nearest untreated neighbor was (1.520, −1.282) at a Mahalanobis distance of 1.383.

Obstacles. The main pitfall was the covariance matrix. `np.cov` treats each *row* as a variable, so passing an N × 2 DataFrame silently returns a 1000 × 1000 matrix. I had to stack Z1 and Z2 into a 2 × N array to get the 2 × 2 covariance, then invert it. A second subtle point: "with replacement" needed no extra code. Each treated row searches all controls and nothing is ever removed, so a control can be reused. For Question 4, I first used the overall largest distance, but "least common support" means the treated row whose *nearest* control is farthest away, so I had to store the *minimum* distance per treated row and then take the maximum.

Finally, the default regularization was a hidden choice I only noticed when I compared it with the unpenalized fit; it turned out to matter very little (2.274 vs 2.268).

Insights. Both methods pulled the estimate down from the naive difference. That's because Z (or Z1 and Z2) raises both the chance of treatment and the outcome, so the treated group would have had higher Y anyway. Reweighting and matching both make the groups comparable on Z, but they answer slightly different questions. IPW with these weights estimates the **ATE** over everyone, while matching treated rows to controls estimates the **ATT**, the effect on those who were actually treated. Checking robustness was useful too: three IPW variants landed between 2.24 and 2.27, which gave me confidence in the result. The overlap check was also reassuring: scores were between 0.05 and 0.93 and the largest weight was about 13, so no single row dominated. Both methods still rely on the same untestable assumption, that the measured Z's capture all the confounding.

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression

df = pd.read_csv('/Users/gkumargaur/workspace/datascience/boston_university/semester3/ms_datascience/data/homework_8.1.csv')

# Step 1: propensity model - Z predicts treatment X (C=np.inf -> no L2 shrinkage)
ps_model = LogisticRegression(C=np.inf).fit(df[['Z']], df['X'])
ps_model_default = LogisticRegression().fit(df[['Z']], df['X'])   # comparison: default C=1.0

# Step 2: propensity scores p = P(X=1 | Z)
df['p'] = ps_model.predict_proba(df[['Z']])[:, 1]
p_default = ps_model_default.predict_proba(df[['Z']])[:, 1]

# Step 3: inverse probability weights
df['ipw'] = np.where(df['X'] == 1, 1 / df['p'], 1 / (1 - df['p']))
w_default = np.where(df['X'] == 1, 1 / p_default, 1 / (1 - p_default))

# Step 4: ATE = weighted mean Y (treated) - weighted mean Y (untreated)
treated, control = df[df['X'] == 1], df[df['X'] == 0]
ate_ipw = (np.average(treated['Y'], weights=treated['ipw'])
           - np.average(control['Y'], weights=control['ipw']))
naive = treated['Y'].mean() - control['Y'].mean()

X, Y, p = df['X'], df['Y'], df['p']
ate_ht = np.mean(X * Y / p) - np.mean((1 - X) * Y / (1 - p))     # unnormalized (Horvitz-Thompson)
t, c = (df['X'] == 1).values, (df['X'] == 0).values
ate_default = (np.average(df['Y'][t], weights=w_default[t])
               - np.average(df['Y'][c], weights=w_default[c]))

print(f"Propensity score range: {df['p'].min():.3f} - {df['p'].max():.3f},  max weight: {df['ipw'].max():.1f}")
print(f"Naive difference in means:                {naive:.3f}")
print(f"IPW ATE estimate (main):                  {ate_ipw:.3f}")
print(f"IPW ATE, unnormalized (Horvitz-Thompson): {ate_ht:.3f}")
print(f"IPW ATE, sklearn default C=1.0:           {ate_default:.3f}")

# Q2: propensity scores of the first three rows
print("\nQ2 - first three propensity scores:", df['p'].head(3).round(3).tolist())
```

Code — Week 8 (IPW ATE from homework_8.1.csv)

```python
from scipy.spatial.distance import mahalanobis

df2 = pd.read_csv('/Users/gkumargaur/workspace/datascience/boston_university/semester3/ms_datascience/data/homework_8.2.csv')

# Q3 Step 1: inverse covariance from ALL rows; np.cov needs a 2 x N matrix (rows = variables)
Z = np.vstack((df2['Z1'], df2['Z2']))          # shape (2, 1000)
inv_cov = np.linalg.inv(np.cov(Z))             # shape (2, 2)

# Step 2: split into treated and untreated
treated2 = df2[df2['X'] == 1]
untreated2 = df2[df2['X'] == 0]
T = treated2[['Z1', 'Z2']].values
C = untreated2[['Z1', 'Z2']].values

# Step 3: nearest untreated row for each treated row (with replacement: controls are never removed)
match_idx, match_dist = [], []
for z_t in T:
    dists = [mahalanobis(z_t, z_c, inv_cov) for z_c in C]
    match_idx.append(np.argmin(dists))
    match_dist.append(np.min(dists))           # kept for Q4
match_dist = np.array(match_dist)

# Step 4: ATT = mean(treated Y - matched control Y)
att = np.mean(treated2['Y'].values - untreated2['Y'].values[match_idx])
print(f"Simple difference in means: {treated2['Y'].mean() - untreated2['Y'].mean():.3f}")
print(f"Matched ATT:                {att:.3f}")

# Q4: least common support = treated row whose NEAREST control is farthest away
worst = np.argmax(match_dist)
print(f"\nQ4 - farthest nearest-match distance: {match_dist[worst]:.3f}")
print("Treated item (Z1, Z2):     ", treated2.iloc[worst][['Z1', 'Z2']].values.round(3))
print("Nearest untreated (Z1, Z2):", untreated2.iloc[match_idx[worst]][['Z1', 'Z2']].values.round(3))
```

Code — Week 8 (Mahalanobis matching ATT from homework_8.2.csv)